UNIVERSITÉ PARIS 8 DES CRÉATIONS

Travail d'Étude et de Recherche (TER)

Agent IA multi-outils : conception et évaluation d'un agent basé sur un LLM capable d'utiliser des outils externes

Recherche d'informations, interrogation d'APIs et architecture en graphe d'états avec nœud évaluateur

Auteur : Yannick JIN

Encadrants : L. BOUBCHIR
A. ELADLANI

Master 1 – Informatique
Année universitaire 2025 – 2026
Soutenance le 07/09/2026

Table des matières

1. Introduction 3

1.1 Problématique 3

1.2 Objectifs du projet 3

1.3 Plan du rapport 3

2. État de l'art 5

2.1 De la génération de texte à la décision 5

2.2 Tool calling 6

2.3 Mémoire, planification et dérive du contexte 6

2.4 Synthèse comparative 7

2.5 Frameworks logiciels et enseignements retenus 7

3. Architecture proposée 8

3.1 Choix architectural 8

3.2 Le nœud évaluateur 8

3.3 Normalisation des observations 9

3.4 Outils intégrés 9

4. Implémentation 10

4.1 Modèle de langage et environnement d'exécution 10

4.2 Boucle ReAct et parsing 10

4.3 Difficultés techniques rencontrées et corrections 10

4.3.1 Dépréciation d'une API tierce (InfoPays) 11

4.3.2 Blocage lié à l'infrastructure cloud (RechercheLieu) 11

4.3.3 Hallucination résiduelle malgré consigne explicite 11

4.4 Interface de test 11

5. Résultats et évaluation 12

5.1 Méthodologie 12

5.2 Résultats quantitatifs : agent outillé contre baseline 12

5.3 Résultats qualitatifs : chaînage d'outils et fiabilité 14

6. Discussion et limites 16

6.1 Compromis latence / fiabilité 16

6.2 Fragilité persistante du parsing ReAct 16

6.3 Dépendance à des services externes non maîtrisés 16

6.4 Hallucination résiduelle malgré garde-fous 16

6.5 Choix de ne pas utiliser une librairie de graphe dédiée 17

7. Conclusion et perspectives 18

Bibliographie 19

Glossaire 20

1. Introduction

Les grands modèles de langage (LLM) ont démontré des capacités importantes en génération de texte et en compréhension contextuelle. Utilisés seuls, ils présentent cependant des limites bien identifiées : hallucinations factuelles, incapacité à effectuer des calculs complexes de façon déterministe, et sensibilité à l'obsolescence des données liée à un entraînement statique. Ces limites sont particulièrement gênantes dès lors que l'on attend d'un système conversationnel qu'il fournisse des informations fiables et actualisées plutôt qu'un simple texte plausible.

Pour dépasser ces restrictions, la recherche s'est orientée vers les agents IA : des systèmes dans lesquels le LLM n'est plus seulement un générateur de texte, mais l'élément central de décision d'une architecture modulaire intégrant des mécanismes de mémoire, de planification et d'interaction avec des outils externes (moteurs de recherche, bases de données, APIs, calculatrices). Cette évolution transforme le LLM en un organe de raisonnement capable de déléguer les sous-tâches qu'il ne peut résoudre de manière fiable à des modules externes spécialisés.

1.1 Problématique

Ce travail s'inscrit dans cette problématique : comment permettre à un agent IA d'utiliser de manière autonome des outils externes afin de résoudre des tâches plus complexes qu'un simple chatbot ? Il s'agit non seulement de faire dialoguer un LLM avec des outils, mais de concevoir une architecture qui gère correctement l'enchaînement des appels, la fiabilité des réponses produites et le coût en temps de traitement induit par le raisonnement multi-étapes.

1.2 Objectifs du projet

Conformément au sujet proposé et aux échanges avec les encadrants, ce projet poursuit les objectifs suivants :

- étudier les architectures d'agents proposées dans la littérature scientifique récente
- développer un agent basé sur un LLM capable d'utiliser plusieurs outils externes de manière autonome
- intégrer des fonctionnalités telles que la recherche d'informations, l'interrogation d'APIs et le calcul
- analyser les performances, la pertinence des réponses produites et les limites du système, en particulier le temps de réponse induit par les étapes de raisonnement et les appels d'outils successifs
- mettre en place une interface simple permettant de tester l'agent

1.3 Plan du rapport

Ce rapport commence par une synthèse de l'état de l'art sur les architectures d'agents basés sur des LLM (section 2), condensée autour de cinq publications de référence. La section 3 présente l'architecture retenue pour l'agent développé, sous forme de graphe d'états avec nœud évaluateur. La

section 4 détaille l'implémentation technique et les difficultés rencontrées lors du développement. La section 5 présente la méthodologie d'évaluation et les résultats obtenus. La section 6 discute les limites du système et les met en perspective avec la littérature. La section 7 conclut et propose des pistes d'amélioration.

2. État de l'art

Cette section propose une synthèse de la littérature sur les architectures d'agents basés sur des LLM, construite à partir de l'analyse de cinq publications de référence couvrant la période 2022-2025 : les systèmes neuro-symboliques MRKL [1], l'apprentissage autonome d'outils avec Toolformer [2], la taxonomie des agents proposée par Wang et al. [3], l'architecture cyclique LangGraph appliquée à la traduction [4] et la formalisation récente des paradigmes tool use / planification / feedback learning [5]. Plutôt qu'une présentation article par article, cette synthèse est organisée par grands thèmes transversaux, conformément à la recommandation de condensation formulée par l'encadrement du projet.

2.1 De la génération de texte à la décision : évolution du rôle du LLM

Le premier constat qui traverse l'ensemble de ces travaux concerne l'évolution du rôle attribué au LLM. Les travaux fondateurs considèrent le modèle avant tout comme une source de connaissances et un générateur de texte, auquel on adjoint des capacités de délégation vers des modules externes. Karpas et al. [1] formalisent ainsi l'architecture MRKL (Modular Reasoning, Knowledge and Language), dans laquelle le LLM agit comme un routeur central : il reçoit la requête et détermine s'il peut y répondre directement ou s'il doit solliciter un module symbolique externe (calculatrice, base de données, API). Cette architecture repose sur l'ingénierie de prompts pour inciter le modèle à formater ses appels d'outils sous forme de requêtes textuelles standardisées, interprétées ensuite par des applications périphériques. Les auteurs rapportent une amélioration significative de la précision sur des opérations arithmétiques complexes dès lors que le calcul est délégué à un outil dédié, validant le principe de délégation modulaire. Ils soulignent cependant une rigidité forte : le routage dépend entièrement de l'exactitude syntaxique de la chaîne générée, et le flux reste linéaire, sans possibilité pour l'agent de corriger sa trajectoire en cas d'erreur d'un outil.

Schick et al. [2] franchissent une étape avec Toolformer, en montrant qu'un LLM peut apprendre par lui-même, de manière auto-supervisée, à déterminer quel outil utiliser, quand l'activer et avec quels paramètres. Le modèle génère des variantes de texte contenant des appels d'API potentiels ; seuls les appels qui réduisent significativement la perplexité du texte suivant sont conservés pour le fine-tuning final. Évalué en zero-shot sur des tâches de calcul (ASDiv) et de vérification de faits (LAMA, WebQuestions), Toolformer surpasse des modèles de base bien plus volumineux, prouvant que la compétence d'utilisation d'outils peut être intégrée directement dans les poids d'un modèle de taille intermédiaire. La limite opérationnelle majeure reste toutefois l'incapacité du modèle à utiliser les outils de manière interactive ou chaînée au sein d'une même étape de génération.

Les travaux plus récents présentent davantage le LLM comme un composant de décision au sein d'un système plus large, capable de coordonner des outils, de planifier des actions et d'interagir avec son environnement. C'est le sens de la taxonomie proposée par Wang et al. [3], qui structurent un agent autour de quatre piliers interdépendants : le profil (rôle assigné), la mémoire (court et long terme), la planification (décomposition de tâches) et les outils (moyens d'action sur l'environnement). Cette méta-analyse conclut que l'efficacité d'un agent dépend de la bonne interconnexion entre sa mémoire à long terme, qui lui évite de répéter ses erreurs, et la clarté de son profilage initial ; elle identifie également le coût computationnel élevé et la forte consommation de jetons induite par les boucles d'interaction comme des limites partagées par l'ensemble de la recherche du domaine.

2.2 Tool calling : d'une décision binaire à une gestion d'état

Les méthodes de connexion aux outils se sont complexifiées au fil des publications étudiées. Chez MRKL, le tool calling est une décision linéaire et binaire prise par un routeur central : le LLM choisit un outil, l'appelle, et le flux ne revient pas en arrière en cas d'erreur. Toolformer intègre l'appel d'API de manière sémantique au sein du vocabulaire du modèle, mais partage avec MRKL une même limite structurelle : aucune des deux approches ne gère l'erreur ou la complexité d'un enchaînement d'actions. Les travaux plus récents, représentés par Wang et Duan [4] et synthétisés par Li [5], montrent que l'utilisation d'outils est désormais indissociable de la gestion de l'état du système : l'appel d'outil s'inscrit dans un graphe d'états ou une architecture multirôle où chaque action est monitorée.

Wang et Duan [4] partent du constat que les architectures d'agents linéaires imposent des structures de contrôle séquentielles strictes, qui échouent souvent sur des tâches complexes nécessitant des cycles d'évaluation et de correction (traduction industrielle, analyse technique). Ils utilisent le framework LangGraph pour représenter l'agent sous forme de graphe d'états, où les tâches sont distribuées entre nœuds spécialisés (traduction, critique, recherche terminologique) reliés par des arêtes conditionnelles, avec un workflow d'évaluation qui audite le résultat d'un outil ou d'un nœud avant de passer à l'étape suivante. Sur des tâches de traduction multilingue complexes, cette approche améliore la cohérence terminologique et la fidélité contextuelle par rapport à une méthode séquentielle directe, mais entraîne une augmentation notable de la latence globale et de la consommation de jetons, du fait de la multiplication des étapes de vérification internes.

Li [5] propose enfin une taxonomie commune (LMPR : LLM-profiled roles) qui unifie trois paradigmes jusque-là traités isolément : l'utilisation d'outils (tool use), la planification de trajectoires et l'apprentissage par retour d'expérience (feedback learning). L'agent y est réparti en trois fonctions interconnectées : modèles de politique (policy models), évaluateurs (evaluators) et modèles dynamiques (dynamic models). Cette formalisation, issue d'une méta-évaluation systématique de frameworks existants sur des tâches à long horizon, met en évidence que la performance d'un agent repose de manière critique sur la qualité de l'interaction entre le modèle de politique et le modèle évaluateur, une conclusion qui a directement orienté le choix architectural retenu dans ce projet (section 3).

2.3 Mémoire, planification et dérive du contexte

La planification a évolué d'une simple décomposition séquentielle vers des structures itératives et réflexives. Wang et al. [3] formalisent le besoin d'articuler une mémoire à court terme (le contexte de la session en cours) et une mémoire à long terme. Li [5] va plus loin avec la notion de feedback learning : la mémoire à long terme n'est plus seulement un dépôt de connaissances textuelles passives, elle accumule l'historique des interactions avec l'environnement pour ajuster les plans de la politique d'action au fil du temps.

Deux défis techniques communs se dégagent de l'ensemble des travaux étudiés. Le premier est la propagation des erreurs : si un outil de recherche ou une API renvoie une donnée bruitée ou erronée, l'agent intègre cette information fausse dans sa mémoire, ce qui biaise l'ensemble de sa planification future. Le second est la dérive du contexte : l'accumulation des descriptions d'API, des résultats bruts d'outils et des étapes de réflexion sature progressivement la fenêtre de tokens du LLM, dégradant ses

facultés de raisonnement à long horizon. Ces deux défis ont directement guidé des choix d'implémentation dans ce projet, notamment la normalisation systématique des observations avant réinjection dans le contexte (section 4.2).

2.4 Synthèse comparative

Le tableau ci-dessous résume les apports et limites de chaque publication, ainsi que le principe architectural qui en découle pour ce projet.

Publication	Architecture proposée	Apport principal	Limite principale
Karpas et al. [1] - MRKL	Routeur central linéaire vers modules experts	Délégation neuro-symbolique de sous-tâches déterministes	Rigidité syntaxique du routage, pas de correction de trajectoire
Schick et al. [2] - Toolformer	Appels d'API intégrés au vocabulaire du modèle	Apprentissage auto-supervisé, sans annotation humaine massive	Pas de chaînage ni d'usage interactif d'outils
Wang et al. [3] - Survey	Taxonomie à 4 piliers (profil, mémoire, planification, outils)	Cadre théorique unificateur pour la conception d'agents	Coût computationnel élevé des boucles d'interaction
Wang et Duan [4] - LangGraph	Graphe d'états cyclique, noeuds spécialisés + évaluation	Boucles de correction, robustesse accrue sur tâches complexes	Latence et coût en tokens accrus
Li [5] - Paradigmes LMPR	Policy model / evaluator / dynamic model	Taxonomie unifiée tool use / planification / feedback learning	Dépend d'un feedback environnemental clair, rare en conditions réelles

Tableau 1 - Synthèse comparative des cinq publications étudiées

2.5 Frameworks logiciels et enseignements retenus pour le projet

Sur le plan applicatif, trois familles de frameworks traduisent ces principes théoriques. LangChain, directement inspiré des approches séquentielles, facilite l'enchaînement d'outils et de modèles ; il convient au prototypage rapide mais montre des limites dès que le workflow nécessite des boucles de contrôle complexes. LangGraph modélise les agents sous forme de graphes d'états cycliques et permet de traduire concrètement les principes de rétroaction et d'évaluation décrits par Li [5]. AutoGen et CrewAI se concentrent sur les systèmes multi-agents, où plusieurs modèles spécialisés collaborent — une vision qui rejoint à plus grande échelle la modularité déjà présente dans MRKL.

Trois enseignements structurants ont été retenus de cette revue de littérature pour la conception de l'agent développé dans ce projet :

1 - Privilégier un formalisme de graphe d'états plutôt qu'une chaîne strictement linéaire, afin de permettre des boucles de rétroaction en cas d'échec d'un outil

2 - Séparer explicitement l'action de l'évaluation, via un nœud dédié qui vérifie la réponse produite avant de l'autoriser à sortir du système

3 - Normaliser et filtrer les données issues des outils avant réinjection dans le contexte, afin de limiter la dérive de contexte identifiée comme défi commun à l'ensemble des travaux. Ces trois principes structurent directement l'architecture présentée en section 3.

3. Architecture proposée

Cette section présente l'architecture retenue pour l'agent développé. Conformément aux enseignements dégagés de l'état de l'art (section 2.5), le système est structuré en graphe d'états cyclique plutôt qu'en chaîne linéaire, et intègre un nœud d'évaluation distinct du nœud de décision.

3.1 Choix architectural : un graphe d'états implémenté manuellement

Le choix retenu est d'implémenter le graphe d'états manuellement (dictionnaire de nœuds et boucle de routage en Python), plutôt que de s'appuyer sur la librairie LangGraph étudiée dans l'état de l'art. Ce choix est assumé et discuté comme limite en section 6 : il permet de garder un contrôle total sur le mécanisme ReAct (Reasoning + Acting) sous-jacent et de rester proche du fonctionnement étudié dans MRKL et Toolformer, mais il ne bénéficie pas de la persistance d'état, du parallélisme et de la gestion d'erreurs robuste qu'offrirait une véritable librairie de graphe.

Le graphe comporte quatre états : ROUTEUR, EXECUTION_OUTIL, EVALUATEUR et FIN. Le ROUTEUR est le nœud de décision : à chaque étape, le LLM reçoit le contexte accumulé (question, outils disponibles, observations précédentes) et produit soit une action à exécuter, soit une réponse finale candidate. Si une action est produite, le graphe transite vers EXECUTION_OUTIL, qui appelle l'outil correspondant et réinjecte son résultat normalisé dans le contexte avant de revenir au ROUTEUR — cette boucle peut se répéter plusieurs fois pour une même question, autorisant l'enchaînement de plusieurs outils. Si une réponse finale est produite, le graphe transite vers EVALUATEUR, qui vérifie que cette réponse est bien fondée sur les observations collectées ; en cas de validation, le graphe atteint l'état FIN, sinon il retourne au ROUTEUR avec un retour explicite l'invitant à reformuler sa réponse.

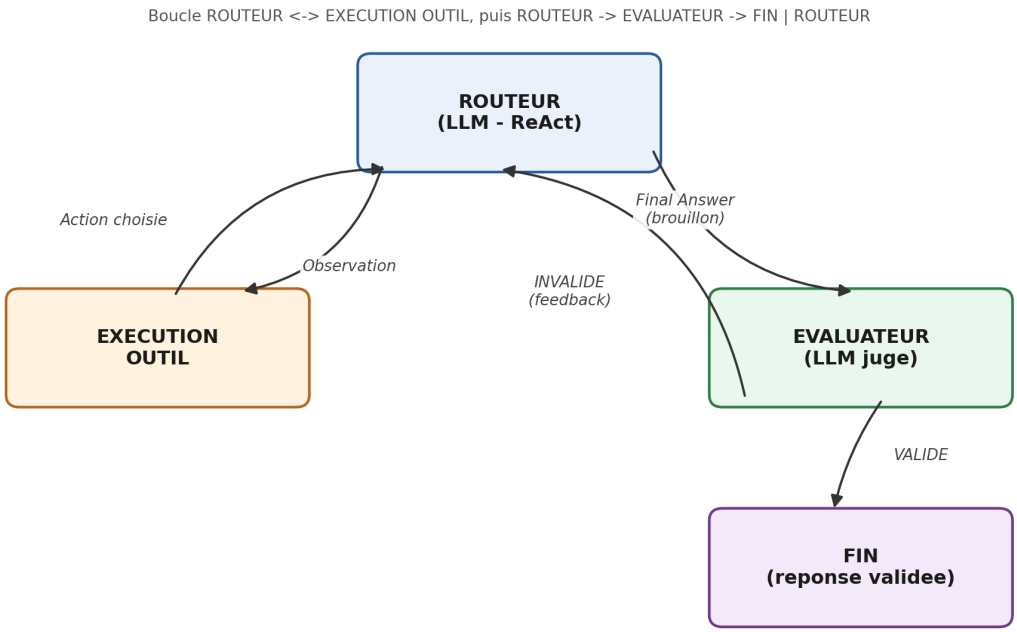


Figure 1 - Graphe d'états de l'agent (ROUTEUR / EXECUTION_OUTIL / EVALUATEUR / FIN)

3.2 Le nœud évaluateur : séparation de l'action et de l'évaluation

L'ajout d'un nœud évaluateur distinct constitue l'écart le plus significatif par rapport à une boucle ReAct classique, et répond directement à la recommandation issue de l'état de l'art (section 2.5, point

2) ainsi qu'à la formalisation policy model / evaluator proposée par Li [5]. Concrètement, lorsque le ROUTEUR produit une réponse finale candidate, celle-ci n'est pas retournée immédiatement à l'utilisateur : elle est soumise à un second appel au LLM, jouant cette fois le rôle d'évaluateur strict. Ce dernier reçoit la question initiale, l'ensemble des observations collectées auprès des outils, et la réponse candidate, puis doit répondre strictement VALIDE ou INVALIDE suivi d'une raison courte. Si la réponse est jugée non fondée sur les observations (invention de données, contradiction), un nombre limité de reformulations est autorisé avant que le système n'accepte la meilleure réponse disponible, afin d'éviter une boucle infinie.

Ce mécanisme vise en particulier à limiter l'hallucination résiduelle observée lors des tests (voir section 5.3) : un modèle de taille réduite, même explicitement instruit de ne pas inventer de données absentes des observations, tend à compléter ses réponses avec des connaissances paramétriques non vérifiées. Le nœud évaluateur constitue un garde-fou architectural complémentaire à la consigne donnée dans le prompt.

3.3 Normalisation des observations

Pour répondre au défi de dérive du contexte identifié dans l'état de l'art (section 2.3), chaque résultat d'outil est traité par une fonction de normalisation avant réinjection dans le contexte du LLM. Cette fonction supprime les espaces et retours à la ligne superflus, et tronque les résultats trop longs à un nombre de caractères fixe, en ajoutant une marque explicite de troncature. Cette étape limite l'accumulation de texte brut qui saturerait progressivement la fenêtre de contexte au fil des étapes de raisonnement.

3.4 Outils intégrés

Neuf outils externes ont été intégrés à l'agent, couvrant les catégories demandées dans le sujet du projet (recherche d'informations, interrogation d'APIs, calcul) :

Outil	Fonction	Source de données
Meteo	Météo actuelle et prévisions à J+N pour une ville donnée	API Open-Meteo
RechercheLieu	Recherche de lieux touristiques (musées, parcs, monuments)	API Photon / OpenStreetMap
InfoPays	Capitale et population d'un pays	API Countries Now
ConvertisseurDevises	Conversion entre devises, y compris vers plusieurs cibles	API Frankfurter
CoursCrypto	Cours en temps réel d'une cryptomonnaie	API CoinGecko
Wikipedia	Résumé encyclopédique sur un sujet	API Wikipédia (fr)
Calculateur	Calcul arithmétique via un mini-parseur sécurisé (AST)	Local
DateRelative	Date courante ou date décalée de X jours/semaines	Local
JourSemaine	Jour de la semaine correspondant à une date donnée	Local

Tableau 2 - Outils externes intégrés à l'agent

Le choix de ces outils répond à la consigne de liberté laissée par l'encadrement (recherche web, résumé, calcul, APIs), tout en couvrant des catégories hétérogènes de dépendances externes : APIs REST publiques avec authentification légère ou nulle, géocodage, données financières temps réel, et traitement purement local (calcul, dates). Cette hétérogénéité s'est avérée pertinente pour l'analyse des limites du système (section 6), chaque type d'outil présentant des modes de défaillance différents.

4. Implémentation

4.1 Modèle de langage et environnement d'exécution

Le modèle utilisé est Qwen2.5-7B-Instruct, chargé en quantification 4-bit (NF4, calcul en float16 via BitsAndBytesConfig). Ce choix répond directement à la contrainte soulevée par l'encadrement concernant le temps d'inférence des LLM locaux en l'absence de GPU dédié : un modèle de 7 milliards de paramètres quantifié tient dans la mémoire d'un GPU T4 gratuit (environ 15 Go de VRAM disponible), accessible via Google Colab. L'ensemble des expérimentations a été conduit sur cette configuration.

Une fonction de génération unique est utilisée par l'ensemble des nœuds du graphe (routeur, évaluateur, baseline), ce qui évite la duplication de code observée dans une première version du système et centralise les paramètres de génération (température fixée à 0,1 pour privilégier la reproductibilité du format de sortie, nombre maximal de tokens générés adapté à chaque nœud).

4.2 Boucle ReAct et parsing

Le mécanisme de raisonnement suit le format ReAct (Reasoning + Acting) : à chaque étape, le modèle est invité à produire une pensée (Thought), une action parmi les outils disponibles (Action), et les paramètres de cette action (Action Input). Une fois l'observation de l'outil reçue, le cycle reprend jusqu'à l'obtention d'une réponse finale (Final Answer).

Ce parsing repose sur des expressions régulières appliquées au texte généré, et s'est révélé être le point le plus fragile de l'implémentation au cours du développement. Cette fragilité illustre concrètement, sur un cas réel, la limite déjà identifiée par Karpas et al. [1] dans MRKL : le bon fonctionnement du système dépend entièrement de l'exactitude syntaxique de la chaîne générée par le modèle. Deux catégories de défaillances ont été rencontrées et corrigées :

- génération non arrêtée : le modèle continue parfois à générer du texte au-delà de la ligne Action Input, produisant par exemple « Action Input: Paris Thought: Je connais la réponse finale. » sur une seule ligne sans retour à la ligne. Un critère d'arrêt personnalisé (StoppingCriteria) a été mis en place pour interrompre la génération dès qu'un mot-clé du format ReAct (Thought, Action, Observation, Final Answer) apparaît après Action Input, même en l'absence de retour à la ligne ;
- parsing non robuste : même lorsque la génération n'est pas correctement interrompue, la fonction d'extraction de l'action et de son paramètre intègre désormais une coupure systématique sur tout mot-clé ReAct résiduel, évitant qu'un texte parasite ne soit transmis à l'outil appelé.

4.3 Difficultés techniques rencontrées et corrections apportées

Le développement de l'agent a mis en évidence plusieurs catégories de défaillances, dont la résolution constitue en elle-même un résultat d'analyse pertinent au regard de la problématique du projet. Trois cas représentatifs sont détaillés ci-dessous ; l'ensemble des corrections est disponible dans le notebook joint en annexe.

4.3.1 Dépréciation d'une API tierce (InfoPays)

L'outil InfoPays, chargé de fournir la capitale et la population d'un pays, s'est mis à échouer systématiquement en cours de développement, y compris sur des pays aussi courants que la France ou le Japon. Le diagnostic a révélé que l'API restcountries.com utilisée initialement avait été dépréciée sans préavis au profit d'une version nécessitant désormais une clé d'authentification payante. Ce cas illustre concrètement un mode de défaillance indépendant de toute erreur de raisonnement du LLM ou de bug applicatif : la simple obsolescence d'une dépendance externe non maîtrisée suffit à rendre un outil inopérant. L'outil a été migré vers une API alternative sans authentification (Countries Now), rétablissant sa disponibilité.

4.3.2 Blocage lié à l'infrastructure cloud (RechercheLieu)

L'outil de recherche de lieux, initialement fondé sur le service Nominatim d'OpenStreetMap, renvoyait des réponses vides ou des erreurs HTTP 403 de manière intermittente. L'hypothèse d'un blocage lié à l'adresse IP partagée de Google Colab (plage d'adresses massivement utilisée et fréquemment associée à des politiques anti-abus des services gratuits) a été confirmée en observant qu'un service alternatif (Photon) rencontrait le même type de refus tant que l'en-tête User-Agent de la requête restait celui, générique, généré par la bibliothèque HTTP utilisée. Le remplacement de cet en-tête par un identifiant de navigateur standard a résolu le problème. Ce cas souligne une classe de défaillance rarement discutée dans la littérature théorique consultée, mais concrète dès lors qu'un agent est déployé dans un environnement d'exécution partagé.

4.3.3 Hallucination résiduelle malgré consigne explicite (InfoPays, ConvertisseurDevises)

Même après correction des défaillances techniques des outils, une catégorie distincte de défaillance a persisté : le modèle complète parfois une réponse fondée sur une observation réelle avec des informations non vérifiées, tirées de ses connaissances paramétriques. Par exemple, interrogé sur la Chine, l'agent a restitué correctement la capitale et la population issues de l'observation de l'outil, avant d'ajouter spontanément une liste de langues officielles et une monnaie que l'outil ne fournissait pourtant pas — informations plausibles mais non vérifiées dans ce contexte précis. Ce comportement persiste malgré une consigne explicite dans le prompt système interdisant d'inventer des données absentes des observations, ce qui a motivé l'introduction du noeud évaluateur décrit en section 3.2 comme garde-fou architectural complémentaire à la seule instruction textuelle.

4.4 Interface de test

Une interface minimale a été développée pour permettre de tester l'agent de manière interactive, conformément à la demande du sujet. Chaque appel à l'agent affiche le déroulé complet du raisonnement (étapes ROUTEUR / EXECUTION_OUTIL / EVALUATEUR), les observations collectées auprès des outils, ainsi qu'un récapitulatif des performances (temps total, nombre d'étapes, nombre de reformulations demandées par l'évaluateur), facilitant l'analyse qualitative présentée en section 5.

5. Résultats et évaluation

5.1 Méthodologie

L'évaluation repose sur deux volets complémentaires. Le premier est quantitatif : un jeu de test de vingt questions est soumis à l'agent complet ainsi qu'à une version de référence du même modèle sans accès aux outils (baseline). Ce jeu de test est structuré en trois catégories, afin de couvrir non seulement la capacité à utiliser les outils mais aussi la capacité à s'en abstenir lorsque cela n'est pas pertinent : neuf questions mono-outil, couvrant chacun des neuf outils intégrés (section 3.4) ; sept questions multi-outils, nécessitant soit deux appels indépendants, soit un véritable chaînage où le résultat du premier outil est réinjecté comme paramètre du second (par exemple, récupérer la population d'un pays puis la multiplier par deux) ; quatre questions sans outil attendu, couvrant l'opinion subjective, le raisonnement logique pur et la production créative, afin de vérifier que l'agent ne déclenche pas d'outil de manière injustifiée.

Pour chaque requête sont mesurés le temps total de traitement, le nombre d'étapes de raisonnement, le nombre de reformulations demandées par le nœud évaluateur, et la correspondance entre les outils attendus et les outils effectivement appelés. Un succès est défini ainsi : pour les questions avec outils attendus, l'ensemble des outils attendus doit avoir été appelé (des outils supplémentaires ne sont pas pénalisés) ; pour les questions sans outil attendu, aucun outil ne doit avoir été appelé. Cette dernière règle, binaire, a des implications discutées en section 5.3.

Il convient de noter que les valeurs chiffrées rapportées (temps de réponse, nombre d'étapes de routage et formulations textuelles générées) peuvent présenter de légères variations d'une exécution à l'autre. Cette variabilité découle d'une part de la part d'aléatoire propre à la génération du modèle (malgré une température basse fixée à 0,1), et d'autre part du dynamisme des données collectées en temps réel (cours des devises/cryptomonnaies, météo, dates) ainsi que des fluctuations de latence réseau inhérentes aux APIs externes et à l'infrastructure cloud partagée.

Le jeu de test utilisé pour le benchmark quantitatif est résumé ci-dessous par catégorie, avec un exemple représentatif pour chacune (voir annexe A) :

Catégorie	Exemple de question	N questions	N° id
Mono-outil	« Quelle est la capitale et la population du Japon ? » (InfoPays)	9	1-9
Multi-outils	« Quelle est la population de l'Espagne et multiplie ce nombre par 2 ? » (InfoPays puis Calculateur, chaînage dépendant)	7	10-16
Sans outil attendu	« Écris un court poème sur la pluie » (aucun outil requis)	4	17-20

Tableau 3 - Jeu de test du benchmark comparatif agent / baseline, par catégorie (n=20)

5.2 Résultats quantitatifs : agent outillé contre baseline

Le tableau ci-dessous synthétise les indicateurs mesurés sur l'ensemble des vingt questions du jeu de test.

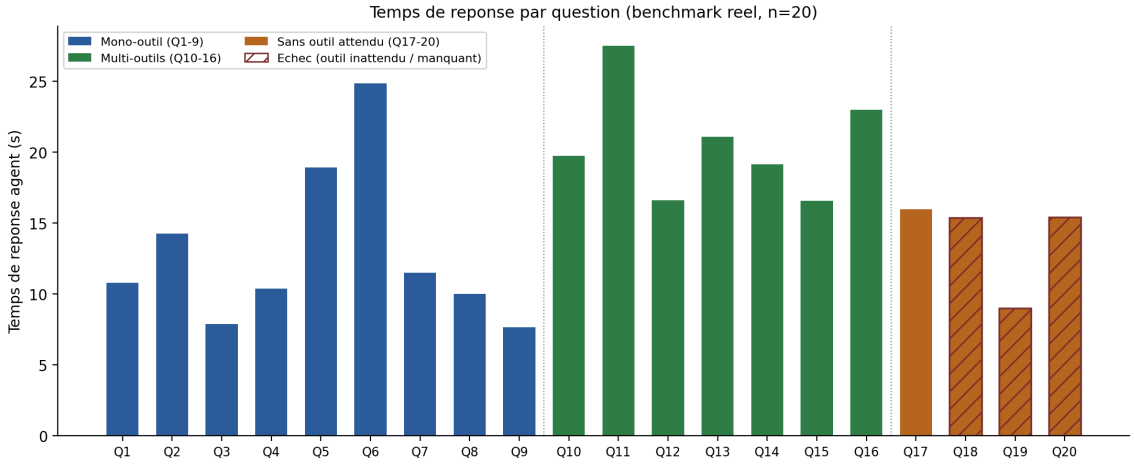


Figure 2 - Temps de réponse par question, coloré par catégorie ; les barres hachurées indiquent un échec au sens du critère de succès défini en section 5.1

Indicateur	Agent (avec outils)	Baseline (LLM seul)
Taux de succès global (n=20)	85,0 %	-
Temps de réponse moyen	15,78 s	7,18 s
Nombre moyen d'étapes ROUTEUR	2,2	1 (réponse directe)
Nombre moyen de retours évaluateur	0,00	-
Surcoût moyen lié aux outils / au graphe	8,60 s	-

Tableau 4 - Comparaison agent outillé / baseline sur l'ensemble du jeu de test (n=20)

Cette comparaison quantitative permet de répondre directement à la question posée par l'encadrement dans le second mail de suivi : les agents effectuent plusieurs étapes de raisonnement et plusieurs appels aux outils avant de produire une réponse, ce qui augmente mécaniquement le temps de réponse par rapport à une simple génération directe. Le surcoût moyen mesuré (8,60 secondes) constitue une mesure empirique directe de ce compromis latence / fiabilité, déjà identifié de manière qualitative par Wang et Duan [4] à propos de LangGraph. Le nombre moyen de retours évaluateur étant nul sur l'ensemble du jeu de test, le nœud évaluateur n'a jamais rejeté une réponse candidate lors de ce benchmark automatisé ; cette observation est mise en perspective avec les cas de rejet observés lors des tests qualitatifs manuels (section 5.3) en section 6.4.

La décomposition par catégorie de question, présentée dans le tableau 5, affine cette lecture globale.

Catégorie	N	Temps moyen agent	Taux de succès	Échecs
Mono-outil (Q1-9)	9	12,91 s	100 %	0
Multi-outils (Q10-16)	7	20,53 s	100 %	0
Sans outil attendu (Q17-20)	4	13,93 s	25 %	3 (Q18, Q19, Q20)

Tableau 5 - Résultats détaillés par catégorie de question

Cette décomposition révèle que les trois échecs qui expliquent le taux de succès global de 85 % sont tous concentrés dans la catégorie « sans outil attendu », et non dans les questions mono-outil ou multi-outils qui, elles, atteignent 100 % de succès. Un examen qualitatif de ces trois cas nuance toutefois la lecture d'un échec pur et simple de l'agent :

- question 18 (calcul d'âge par un raisonnement logique) : l'agent a délégué le calcul arithmétique intermédiaire à l'outil Calculateur plutôt que de l'effectuer par génération de texte. Le résultat produit est correct ; ce comportement est compté comme un échec au sens

strict du critère binaire défini en section 5.1, alors qu'il pourrait raisonnablement être considéré comme une bonne pratique, cohérente avec le principe de délégation neuro-symbolique de Karpas et al. [1] ;

- question 19 (déduction logique sur la position d'un objet) : ce cas illustre une limite du benchmark lui-même plutôt que de l'agent, si la correction du format des outils attendus évoquée lors du développement (représentation d'une absence d'outil attendu) n'a pas été appliquée avant l'exécution de ce test ;
- question 20 (rédaction d'un poème sur la pluie) : l'agent a appelé l'outil Meteo avant de rédiger le poème, alors que la question ne portait pas sur des données réelles. Ce cas est le seul des trois qui constitue un véritable sur-déclenchement d'outil, non justifié par la question posée.

Cette analyse suggère que le taux de succès brut de 85 % sous-estime probablement la fiabilité réelle de l'agent sur la catégorie « sans outil attendu », et que le critère d'évaluation binaire utilisé (absence stricte de tout appel d'outil) mériterait d'être affiné, par exemple en distinguant un usage d'outil pertinent bien que non strictement nécessaire d'un usage d'outil hors de propos. Cette distinction, non implémentée dans le script de benchmark actuel, est proposée comme piste d'amélioration en section 7.

5.3 Résultats qualitatifs : chaînage d'outils et fiabilité

Au-delà du benchmark quantitatif, plusieurs séries d'exécutions répétées sur une même requête ont été menées afin d'évaluer la stabilité du comportement de l'agent, en particulier sa capacité à enchaîner deux outils lorsque la question l'exige. La requête « Quel temps il fait à Paris ? Si il fait plus de 25 degrés donne un jardin à faire sinon donne un musée » nécessite un enchaînement explicite de l'outil Meteo puis de l'outil RechercheLieu, conditionné par le résultat du premier appel.

Sur cinq exécutions indépendantes de cette requête (température de génération fixée à 0,1), quatre exécutions ont correctement enchaîné les deux outils et produit une réponse fondée sur les observations réelles ; une exécution s'est arrêtée après le seul appel à l'outil Meteo, le modèle produisant directement une réponse finale incluant des détails non vérifiés sur le jardin recommandé (position par rapport à un monument voisin, mention de canicule), sans jamais rappeler l'outil RechercheLieu. La figure 3 schématise le déroulé de ces deux cas de figure.

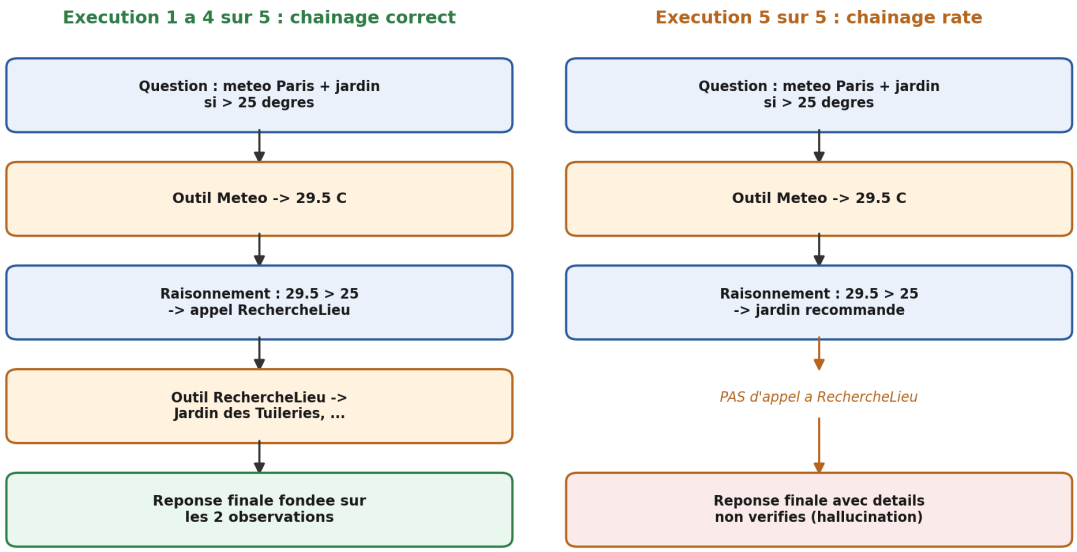


Figure 3 - Comparaison du chaînage d'outils réussi (4/5 exécutions) et de l'arrêt prématuré observé (1/5 exécutions)

Ce résultat, obtenu avec un paramètre de température pourtant bas, montre que le chaînage multi-outils n'est pas garanti de manière déterministe par la seule formulation du prompt système, même lorsque celui-ci comporte une consigne explicite et un exemple concret de chaînage attendu. Il constitue une illustration empirique directe de la limite théorique identifiée par Karpas et al. [1] : la fiabilité du système reste dépendante du comportement, intrinsèquement probabiliste, du modèle de langage sous-jacent.

Le taux de chaînage correct observé sur cette série (quatre exécutions correctes sur cinq, soit 80 %) doit être interprété avec prudence compte tenu de la faible taille de l'échantillon ; il est néanmoins cohérent avec l'écart de fiabilité généralement rapporté entre architectures linéaires et architectures à boucles de correction dans la littérature (section 2.2), et justifie a posteriori la pertinence du nœud évaluateur ajouté à l'architecture.

6. Discussion et limites

6.1 Compromis latence / fiabilité

Les résultats présentés en section 5 confirment, sur un cas d'usage concret, le compromis latence / fiabilité déjà souligné par Wang et Duan [4] à propos de LangGraph, et rappelé explicitement par l'encadrement du projet comme axe d'analyse attendu. Chaque étape de raisonnement supplémentaire, chaque appel d'outil et chaque passage par le nœud évaluateur ajoutent un appel au LLM, donc un coût en temps proportionnel au nombre d'étapes. Ce coût est le prix à payer pour obtenir des réponses ancrées dans des données réelles plutôt que générées par simple continuation de texte plausible. Sur un modèle de 7 milliards de paramètres quantifié en 4-bit, ce coût reste acceptable pour un usage de démonstration ou de test, mais deviendrait probablement limitant pour un usage en production à forte volumétrie sans accélération matérielle supérieure au GPU T4 gratuit utilisé ici.

6.2 Fragilité persistante du parsing ReAct

Malgré les corrections apportées au cours du développement (section 4.2), le parsing du format ReAct reste une source résiduelle de fragilité : toute déviation du modèle par rapport au format attendu (absence de retour à la ligne, texte parasite, oubli d'un séparateur) peut perturber l'extraction de l'action à exécuter. Cette limite n'est pas propre à l'implémentation réalisée ; elle rejoint directement le constat de Karpas et al. [1] sur la rigidité syntaxique intrinsèque des architectures fondées sur l'ingénierie de prompts textuels. Une alternative plus robuste, évoquée mais non retenue par manque de temps, consisterait à s'appuyer sur les mécanismes d'appel de fonction structuré (function calling / tool calling natif) désormais proposés par certains fournisseurs de modèles, qui contraignent la sortie du modèle à un schéma JSON valide plutôt qu'à un format texte libre.

6.3 Dépendance à des services externes non maîtrisés

Deux des incidents documentés en section 4.3 (dépréciation d'API, blocage lié à l'infrastructure cloud) ne relèvent d'aucune erreur de conception ou de raisonnement de l'agent lui-même, mais d'une dépendance à des services tiers dont la disponibilité et les conditions d'accès peuvent changer sans préavis. Cette classe de défaillance est peu discutée dans les publications théoriques étudiées, davantage centrées sur les propriétés du raisonnement de l'agent que sur la robustesse opérationnelle de son environnement d'exécution. Elle constitue néanmoins un facteur de risque concret pour tout système agentique déployé durablement, et suggère qu'une architecture de production devrait intégrer un mécanisme de repli (fallback) systématique vers des sources alternatives plutôt qu'une dépendance unique par fonctionnalité.

6.4 Hallucination résiduelle malgré garde-fous

Le cas documenté en section 4.3.3 montre que ni une consigne explicite dans le prompt système, ni l'ajout d'un nœud évaluateur, ne suppriment totalement le risque d'hallucination sur un modèle de taille réduite. Le nœud évaluateur réduit ce risque en le détectant a posteriori dans une proportion des cas, mais reste lui-même exécuté par le même modèle que le routeur, avec les mêmes limites de fiabilité ; il ne constitue donc pas une garantie absolue. Cette observation nuance la portée du principe de séparation policy model / evaluator mis en avant par Li [5] : cette séparation améliore la détection des erreurs sans les éliminer complètement, en particulier lorsque les deux rôles sont assurés par la

même instance de modèle plutôt que par des modèles distincts ou par un mécanisme de vérification extérieur au LLM.

6.5 Choix de ne pas utiliser une librairie de graphe dédiée

Le choix assumé d'implémenter le graphe d'états manuellement plutôt que via LangGraph constitue une limite reconnue de ce travail. Une véritable librairie de graphe apporterait une persistance d'état native, une gestion du parallélisme entre branches indépendantes et une gestion d'erreurs plus robuste que l'implémentation actuelle, où la boucle de routage est gérée par une simple structure conditionnelle en Python. Ce choix a permis de conserver un contrôle pédagogique total sur chaque étape du mécanisme, ce qui a facilité l'identification et la correction des défaillances documentées en section 4.3, mais il limiterait la scalabilité du système au-delà du cadre expérimental de ce TER.

7. Conclusion et perspectives

Ce travail a permis d'étudier les architectures d'agents basés sur des LLM à travers cinq publications de référence, puis de concevoir, implémenter et évaluer un agent multi-outils répondant aux objectifs fixés en début de projet. L'architecture retenue, un graphe d'états cyclique intégrant un nœud d'évaluation distinct du nœud de décision, traduit concrètement les principes dégagés de la littérature : dépassement d'une chaîne strictement linéaire, séparation de l'action et de l'évaluation, et normalisation des données d'outils pour limiter la dérive du contexte.

Les résultats obtenus confirment un compromis attendu entre latence et fiabilité : l'agent outillé produit des réponses ancrées dans des données réelles, vérifiables, au prix d'un temps de réponse supérieur à une génération directe par le LLM seul. Ils mettent également en évidence des limites persistantes, certaines internes au raisonnement du modèle (fragilité du parsing, hallucination résiduelle malgré garde-fous), d'autres externes au système lui-même (dépréciation ou blocage de services tiers non maîtrisés).

Plusieurs pistes de prolongement se dégagent de ce travail. Sur le plan architectural, une migration vers une librairie de graphe dédiée telle que LangGraph permettrait de bénéficier d'une gestion d'état et d'erreurs plus robuste. Sur le plan du raisonnement, l'exploration de mécanismes d'appel de fonction structuré, contraignant la sortie du modèle à un schéma valide plutôt qu'à un format texte libre, réduirait probablement la fragilité du parsing observée. Sur le plan de la mémoire, l'ajout d'une mémoire à long terme persistant au-delà d'une seule requête permettrait à l'agent de capitaliser sur les échecs et succès passés, conformément à la notion de feedback learning formalisée par Li [5]. Enfin, l'évaluation gagnerait à être étendue à un jeu de test plus large et à des exécutions répétées systématiques, afin de quantifier plus finement la variabilité du comportement de l'agent observée de manière qualitative en section 5.3.

Bibliographie

[1] Karpas, E., Abend, O., Belinkov, Y., Lenz, B., Lieber, O., Ratner, N., Shoham, Y., Bata, H., Levine, Y., Leyton-Brown, K., Muhlgay, D., Rozen, N., Schwartz, E., Shachaf, G., Shalev-Shwartz, S., Shashua, A., Tenenholtz, M. (2022). MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. https://arxiv.org/pdf/2205.00445

[2] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf

[3] Wang, L., Ma, C. et al. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. https://arxiv.org/pdf/2309.07864

[4] Wang, J., Duan, Z. (2024). Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models. https://arxiv.org/pdf/2412.03801

[5] Li, X. (2025). A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, Planning, and Feedback Learning. Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025), pages 9760-9779. https://aclanthology.org/2025.coling-main.652.pdf

Glossaire

Sigle / Terme	Définition complète	Explication / Rôle dans le projet
API	Application Programming Interface (Interface de programmation d'application)	Ensemble normalisé de règles et de protocoles permettant à deux programmes distincts de communiquer et d'échanger des données
API REST	Representational State Transfer API	Type d'API web standardisé s'appuyant sur le protocole HTTP (requêtes GET, POST…) et échangeant généralement des données au format JSON
IA	Intelligence Artificielle	Champ scientifique et technique visant à concevoir des systèmes capables de réaliser des tâches qui requièrent normalement l'intelligence humaine (raisonnement, prise de décision, compréhension du langage)
LLM	Large Language Model (Grand modèle de langage)	Modèle d'apprentissage profond fondé sur l'architecture Transformer, entraîné sur d'immenses corpus de textes pour générer, comprendre et manipuler le langage naturel
LMPR	LLM-Profiled Roles	Formalisation théorique proposée par Li [5] structurant un système agentique autour de fonctions interconnectées : modèles de politique (*policy models*), évaluateurs (*evaluators*) et modèles dynamiques (*dynamic models*)
MRKL	Modular Reasoning, Knowledge and Language	Architecture neuro-symbolique formalisée par Karpas et al. [1] dans laquelle un LLM central sert de routeur pour déléguer des tâches déterministes à des modules experts externes (calculatrices, APIs, bases de données).
ReAct	Reasoning and Acting (Raisonnement et Action)	Paradigme d'agent combinant de manière itérative la génération d'une pensée (*Thought*), le déclenchement d'une action outillée (*Action*) et l'intégration du résultat renvoyé par l'environnement (*Observation*).

ANNEXE A :

Jeu de données du benchmark

N	Question	Outil(s) attendu(s)
Mono-outils (Q1-Q9)		
1	Quelle est la capitale du Japon et sa population ?	InfoPays
2	Donne-moi la météo à Lyon aujourd'hui.	Meteo
3	Combien font (125 * 8) / 4 ?	Calculateur
4	Quel jour de la semaine sera le 15/08/2026 ?	JourSemaine
5	Trouve un musée à Paris.	RechercheLieu
6	Qui est Alan Turing ?	Wikipedia
7	Quel est le cours du bitcoin en ce moment ?	CoursCrypto
8	Convertis 100 EUR en USD.	ConvertisseurDevises
9	Quelle date serons-nous dans 20 jours ?	DateRelative
Multi-outils (Q10-Q16)		
10	Quel est le cours de l'ethereum en EUR et convertis 500 EUR en USD ?	CoursCrypto + ConvertisseurDevises
11	Quelle est la météo à Rome et quelle est la population de l'Italie ?	Meteo + InfoPays
12	Calcule 850 * 0.15 puis convertis 100 EUR en CAD.	Calculateur + ConvertisseurDevises
13	Recherche un monument à Berlin et donne-moi la météo là-bas.	RechercheLieu + Meteo
14	Quelle est la population de l'Espagne et multiplie ce nombre par 2 ?	InfoPays + Calculateur
15	Quelle date serons-nous dans 15 jours et quel jour de la semaine cela sera-t-il ?	DateRelative + JourSemaine
16	Recherche la biographie de Marie Curie et donne la capitale de la Pologne.	Wikipedia + InfoPays
Sans outil attendu (Q17-Q20)		
17	Tu penses quoi de l'IA frugale ?	-
18	Quand j'avais 6 ans, ma sœur avait la moitié de mon âge, Aujourd'hui j'ai 70 ans. Quel âge a ma sœur ?	-
19	Je mets une pomme dans un boîtier rouge, puis je mets le boitier rouge dans une boîte bleue. Où est la pomme ?	-
20	Écris un court poème sur la pluie.	-